

Mizan.ai

Frontend Structure & Page-by-Page Reference

How the frontend is built, what every page does, and the UX patterns established while building it — written directly from the current state of frontend/ in the repo.

1. Overview

The frontend is plain HTML, CSS, and JavaScript — no framework, no build step, no bundler. This was the user's explicit, standing choice for this project, not a placeholder pending a “real” framework later. Every page is a single self-contained `.html` file that loads one shared stylesheet and two small shared scripts.

- **No external dependencies** — no CDN scripts, no web fonts, no icon libraries. Fonts are system-native (`-apple-system`, Segoe UI, Helvetica Neue, Tahoma, Arial); icons are inline SVG or emoji. The whole frontend works fully offline once loaded.
- **Bilingual by design** — Arabic content uses `dir="rtl"` wrapper elements and the CSS deliberately avoids hardcoding left/right (uses logical properties like `margin-inline-start`) so RTL sections flip correctly without extra overrides.
- **Light + dark themes** — entirely automatic via `prefers-color-scheme`, no manual toggle. Every color is a CSS custom property with a light and dark value defined once in `style.css`.
- **One design system, one file** — every page pulls from the same `style.css`, so a component (a card, a button, a table, a chat bubble) looks identical everywhere it's used.

How pages are served

There is no frontend build/deploy pipeline yet — pages are served as static files (in local development, a plain `python -m http.server` in `frontend/`, or VS Code's Live Server extension; both are already in the backend's CORS allow-list). Each backend feature is its own microservice on its own port, and the frontend talks to each directly from the browser — there is no API gateway/proxy layer in front of them for the frontend to go through.

2. Shared infrastructure

2.1 config.js — service URLs

A single object, `MIZAN_CONFIG`, mapping each backend service the browser talks to directly (chatbot, calculator, translator, converter, explainer, editor, comparator) to its base URL. This is the only place a URL is hardcoded — every page reads from here instead of hardcoding its own. `doc-extraction` is deliberately absent: it's only ever called service-to-service (by Calculator/Comparator's backends), never directly from the browser.

2.2 common.js — auth & request helpers

Shared by every page. Authentication is a JWT bearer token (not a cookie), issued by `/api/auth/login` or `/api/auth/register`, stored in `localStorage`. There is no logout endpoint — logging out is simply discarding the token client-side (stateless JWT).

- `mzRequireAuth()` — called at the top of every page that needs a logged-in user. Redirects to `login.html` immediately if no token is stored. It does not validate the token locally (expiry/signature) — that's left to the backend on the next real request.
- `mzFetch(url, options)` — wraps `fetch()`, attaching the `Authorization: Bearer ...` header automatically. On a 401 response, it clears the stored token and redirects to login itself, so no individual page has to special-case an expired session.
- `mzErrorDetail(response)` — extracts the `detail` field FastAPI's default error shape returns, falling back to the HTTP status text. Used everywhere a failed request needs to show the real backend error message instead of a generic one.

2.3 style.css — the design system

One stylesheet, ~560 lines, used by every page. Built around CSS custom properties so the whole palette can be reasoned about (and re-themed) from one place.

Token	Light	Dark	Used for
--accent	#0e7c66	#2bb894	Primary brand green — headings, links, focus rings, primary buttons
--bg / --surface	#f4f6f5 / #ffffff	#101513 / #171e1b	Page background vs. card/panel background
--surface-2	#eef2f0	#1e2622	Subtle shaded panels (e.g. Comparator's "Needs review" section, table header rows)
--text / --text-secondary / --text-muted	3-step gray scale	3-step gray scale	Primary copy / labels / captions
--error / --error-soft	#c22b1f / light red	#ef6b5f / dark red	Error banners
--radius / --radius-sm	12px / 8px	same	Card corners / button-and-input corners

Full token list lives at the top of style.css — this table covers the ones that show up repeatedly across pages.

Key reusable components

Class / selector	What it is
.mz-shell	Max-width, centered page container (1040px)
.mz-topbar	Full-width header bar with the logo and nav links (About / Contact / Log out) — only on app.html
.mz-card (.wide / .xwide)	The standard content panel — white surface, thin border, 3px accent-colored top border, rounded corners, soft shadow. Capped at 720px by default; .wide (860px) and .xwide (980px) opt into more room for chat-heavy pages
dialog.mz-card	A native <dialog> styled as a card, used for Comparator's review popup — browser-native modal semantics (focus trap, Esc-to-close, top-layer stacking) instead of a hand-rolled overlay
.mz-grid / .mz-feature-card	The home page's feature-picker grid — auto-fit responsive columns, each card an icon + label, with a .soon variant (dimmed, non-clickable) for not-yet-built features
.mz-table	Standard data table — shaded uppercase header row, row hover highlight, used for Calculator's line items and Comparator's matched-pairs table
.mz-chat-log / .mz-chat-msg	Sender-aligned chat bubbles (assistant left, user right — flips correctly under RTL via flexbox justify-content, not margins) plus a bouncing-dots typing indicator shown only while a request is actually in flight
.mz-status / .mz-error	Muted informational text vs. a red error banner — the two states almost every page's async actions resolve to
button.mz-secondary	Outline/ghost button variant for a lower-emphasis secondary action, next to the default solid-green primary button style

3. Page inventory

Every file in frontend/, what it's for, and which backend service(s) it calls.

File	Purpose	Auth?	Backend service(s)
login.html	Sign in	No	chatbot (auth endpoints)
register.html	Create an account	No	chatbot
forgot-password.html	Request a temporary password	No	chatbot
reset-password.html	Set a real password after a temp-password login	Yes (limited token)	chatbot
app.html	Home — feature grid + always-on default chat	Yes	chatbot
about.html	Static bilingual about page	Yes	—
contact.html	Feedback form	Yes	chatbot
know-vat-zatca.html	Grounded VAT/ZATCA Q&A chat, with citations	Yes	explainer
translator.html	Paragraph EN<->AR translation	Yes	translator
calculator.html	VAT compliance calculator (5-step) + past-filings Q&A	Yes	calculator
comparator.html	Document reconciliation (SAP/Odoo export vs. a second document)	Yes	comparator
converter.html	Format conversion + all PDF-editing operations, incl. signatures	Yes	converter, editor

4. Auth pages

Four small, near-identical pages sharing one visual shell (`.mz-auth-shell` — centered, narrow, logo-first). All talk to the chatbot service's auth endpoints directly.

- **login.html** — email + password. On success, stores the returned JWT. If the backend flags `must_reset_password` (a temporary-password login), it's routed straight to `reset-password.html` instead of the home page; that limited token is rejected by every other endpoint until a real password is set.
- **register.html** — name (optional) + email + password. Logs the new user in immediately on success (no separate email-verification step in this flow).
- **forgot-password.html** — email only. Notably, the backend always returns the exact same generic confirmation message whether or not that email is actually registered — a deliberate security choice (never let this endpoint reveal which emails have accounts) that the frontend just displays verbatim.
- **reset-password.html** — new password only (the user is already authenticated via the limited must-reset token at this point). On success, the returned fresh full-access JWT replaces the limited one.

Both `login.html` and `register.html` check `mzGetToken()` on load and redirect straight to `app.html` if already signed in, so a logged-in user can't land back on these forms via a stale bookmark.

5. Home, About, Contact

5.1 app.html — home

The top bar (logo, English wordmark with an Arabic ■ ■ ■ ■ ■ mark centered independently, About/Contact/Log out links) appears only here. Below it: a hero title in English and Arabic, then the feature grid — five real, clickable feature cards (VAT & ZATCA Guidance, Document Translator, Compliance Calculator, Document Reconciliation, Convert & Edit Files) plus two dimmed, non-clickable “Coming soon” cards (Multi-Document Synthesis, Voice Assistant).

Below the grid sits a general-purpose chat box, available by default with no click-through — this is the one always-visible AI entry point on the site. It's tied to a `LangGraph thread_id` the backend mints on the first message; reloading the page starts a fresh conversation (the thread isn't persisted client-side on purpose).

First-message UX: the underlying model container can be cold on the very first message of a session (up to roughly 60–90 seconds). Rather than the plain three-dot typing indicator looking like the app has hung, the first message only shows an explicit “Getting things ready, please wait...” notice ahead of the dots. Every message after that just gets the quick dots. This exact pattern (an `isFirstMessage` flag, only ever true once per page load) is reused identically on `know-vat-zatca.html`.

5.2 about.html / contact.html

`about.html` is a static bilingual page (full Arabic block, then an `<hr>`, then the equivalent English content) describing the product and its data-privacy stance — no backend calls at all. `contact.html` is a single feedback textarea that POSTs to the chatbot service's feedback endpoint and shows a plain “Sent — thank you.” confirmation.

6. Chat features

6.1 know-vat-zatca.html

A dedicated, full-width chat page (uses `.mz-card.xwide` — nothing else on the page competing for space, so it gets the tallest chat log of any page). Functionally near-identical to the home page's chat (same warming-up first-message pattern, same bubble styling), with one key difference: every assistant reply that includes citations renders them inline under the reply text — document type, jurisdiction, and version label, joined for however many source chunks backed that answer. This is the one page where the AI's answer is explicitly grounded and sourced, and the frontend surfaces that grounding rather than just showing the reply text.

6.2 translator.html

The simplest feature page — one textarea, one button. Language is auto-detected server-side and always translated to “the other” language (no manual language picker). Result shows the translated text in a second, read-only textarea labeled with the detected direction (e.g. “English → Arabic”). A “Translate a file” option is shown as a dimmed “Coming soon” card below the form.

7. calculator.html — VAT compliance calculator

The most stateful page after Comparator. A five-step flow, each step revealed only once the previous one succeeds, plus an independent search box that doesn't fit the linear flow.

Step	What happens	Endpoint
1. Upload documents	Upload the source file(s) for a filing	POST /api/compliance/upload
2. Extracted fields	Shows what was automatically pulled from the upload	GET /api/compliance/{id}/extraction
3. Line items	Editable line-item table — add/remove rows, each with a description, amount, and AI-suggested tax category the user reviews	— (client-side editing)
4. Filing details & confirm	“Check compliance” runs a dry-run structural validation only; “Confirm & calculate” runs the real structural validation + full calculation + AI narration together — the one point where what's on screen becomes final	POST /api/compliance/{id}/validate (twice — approved=false then true)
5. Download report	Renders and downloads the final filing report	POST /api/compliance/{id}/report

A separate, always-visible “Ask about past filings” search box sits near the top of the page, independent of the 5-step flow — a plain-language Q&A over the user's own filing history (POST /api/compliance/filing-notes/qa). If saving a filing note fails silently in the background, a small inline “Filing note wasn't saved” notice with its own Retry button appears rather than failing the whole flow.

Reset (“Start over”) button lives inside the “1. Upload documents” card, bottom-right — placed there deliberately after user feedback, rather than at the top or bottom of the whole page. It clears the file selection and every later step's state, but explicitly does *not* clear the independent “Ask about past filings” box, since that's a separate concern.

8. converter.html — Convert & edit

A category drill-down, not a single form: a top-level choice between “Convert doc to PDF” and “Edit PDF”, the latter opening a sub-menu of individual PDF operations. Navigation is pure show/hide of `.mz-panel` elements driven by `data-target` attributes on cards and back-links — no routing, no history stack, exactly one panel visible at a time.

Every operation shares one helper, `runFileOperation(url, formData, prefix, downloadName)`: POST the form data, show a “Working...” status while in flight, download the returned file on success, or show the real backend error message on failure — a “Working” text status here rather than the busy-button pattern used elsewhere on this page, since several of these panels don't have a single obvious button to relabel.

Panel	Backend service	Notes
Convert DOCX/XLSX to PDF	converter	PDF-to-DOC, PDF-to-XLSX, and scanned-doc-to-PDF are shown as dimmed “Coming soon” cards
Merge	editor	2+ PDFs, combined in upload order
Split	editor	0-indexed start/end page range
Rotate	editor	“all” or a comma-separated page list, angle must be a multiple of 90
Watermark	editor	Free-text watermark string
Password-protect	editor	Sets the password required to open the file
Compress	editor	No extra options
Add signature	editor	See below — the most involved panel on this page

8.1 Add signature — the detailed panel

This panel folds in what was originally scoped as a separate “PDF Editor” feature — it has no page of its own; by design it lives here even though it's served by a different backend service (editor, not converter).

- **Page preview:** “Load page preview” fetches a rendered PNG of the chosen page (POST `/api/pdf-editor/page-preview`), which returns both the image and the page's real point-dimensions via response headers.
- **Click or drag to place:** clicking the preview drops a default-sized placement box at that spot; clicking and dragging draws a custom-sized box. A small handle at the box's corner allows resizing after the fact. Screen-pixel coordinates are converted to real PDF point-coordinates using the ratio between the displayed image size and the page's actual dimensions from those response headers.
- **Manual override:** the resulting `x0, y0, x1, y1` placement is also shown (and directly editable) as plain text, for anyone who'd rather type exact coordinates.
- **Darkness slider:** a 0.4–2.5 range control (contrast adjustment, not brightness — chosen after verifying that naive brightness scaling doesn't actually change what the background-removal algorithm detects) to help with signature photos that come out too faint or too heavy.
- **Reset** button clears the file, page number, preview state, placement box, and darkness value together.

The panel's own inline guidance text sets expectations up front: sign on a plain, evenly lit sheet of paper, photographed straight-on, with the ink clearly darker than the paper — since the background-removal is a deterministic color-distance threshold, not a trained model.

9. comparator.html — Document reconciliation

The most developed page on the site, and the one that has gone through the most rounds of real user testing and iteration. Reconciles a company's own accounting export against a second document (a bank statement, a vendor statement, etc.) — one relationship, one pair of files, per run.

9.1 Flow

Step	What happens
1. Upload both documents	"Our records" + "Their statement" file inputs, plus optional date-proximity and amount-tolerance overrides. Clicking Reconcile runs the matching engine's Passes 1-4 and opens a popup with the results
Review popup	A native <dialog> showing the clean Matched table, a collapsible "we mapped your columns" panel, and a shaded "Needs review" section with one card per item needing a human decision
2. Download report	Appears once the popup's "Confirm & generate" succeeds. Holds the AI-generated plain-language summary (if produced), the optional "Check for more possible groupings" button, and the format picker + download button

9.2 Detected-columns panel

Real-world exports name their columns very differently from one company or bank to the next, so the backend guesses the mapping via a synonym list rather than a hardcoded schema. Rather than trust that guess silently, it's shown to the reviewer as a collapsed-by-default <details> block ("We mapped your columns — tap to check") so it doesn't compete for space with the actual review items, but is one click away. Inside: two small two-column tables ("Our field" / "Found as"), one per side, covering Reference/Date/Amount/Description.

9.3 The Needs Review cards

Every review item gets its own card, with wording and controls specific to why it needs a decision. All of them share the same Decision control (Confirmed / Dismissed, or left as-is meaning unresolved) plus an optional free-text explanation.

Review kind	What it means	Card control
Possible match	Amount+date proximity found exactly one candidate, but no reference match — less certain than a clean match	Decision dropdown only
Multiple candidates	Row_a has genuinely more than one plausible row_b	Radio buttons, one per real candidate
Multiple candidates — contested	Two or more of OUR records are competing for the SAME single candidate on their side (e.g. two payments, one bank receipt)	Merged into ONE card: radio choice between the competing records, plus explicit "None of these" and "Not sure yet — leave unresolved" options
Amount mismatch	Same reference on both sides, but the amounts disagree	Decision dropdown; if the gap is close to 15%, the card explicitly suggests a VAT-inclusive/exclusive recording difference instead of a generic error

Review kind	What it means	Card control
Group match (consolidated)	2-4 items on one side sum to one item on the other, found via a real connecting signal (date, description, or sequential reference)	Lists every grouped item plus the summed total; decision dropdown
Possible grouping (amount only)	Same shape as Group match, but found by amount alone with no connecting signal — weaker evidence, only ever produced by the manually-triggered “Check for more possible groupings” pass	Same as Group match, but visually distinct: an amber left border and explicitly-worded warning text, so it's never confused with the stronger Group match evidence

9.4 “Check for more possible groupings”

A second, optional matching pass for partial/installment payments that share no date, description, or reference signal at all with what they're settling — exactly the case the signal-gated Group match pass can't catch. Deliberately gated behind its own button on the Download report step (only reachable after a full first review-and-confirm cycle), never run automatically or in parallel with the first review. Clicking it re-opens the same review popup, showing only the newly-found items; confirming them reuses the exact same Confirm & generate flow.

9.5 Reset

Sits next to the Reconcile button inside the upload card (moved there after user feedback, rather than living at the top of the page). Clears the file selections, closes the review popup if open, hides the report step, and clears every piece of state the flow accumulated — matched pairs, review items, narration text, and the column-mapping panel.

10. Patterns established across the whole frontend

None of these are written down in one place in the code — they emerged page by page, through real user testing and feedback, and are now consistent across every page that needed them.

- **Busy-button pattern** — a shared `withBusyLabel(btn, busyText, fn)` helper disables a button and swaps its label (e.g. “Reconcile” → “Matching...”) for the duration of an async action. An honest “still working” signal, not a fake staged-progress animation — the frontend has no way to know which internal stage a plain request/response call is actually in, so it doesn’t pretend to.
- **Reset buttons live where the input is, not at page extremes** — both Calculator’s and Comparator’s reset buttons ended up placed inside or next to the specific card they reset, after early top-of-page or bottom-of-page placements were explicitly rejected in user feedback.
- **Green is the only button color** — an early attempt at a more “conventional” light blue for buttons was explicitly reverted after feedback that it didn’t match the rest of the page’s theme. Every primary action button uses the same brand-green - -accent color; .mz-secondary is the only intentional variant, for lower-emphasis actions.
- **First-message warming-up notice, never a wrong time estimate** — both chat pages show an explicit “Getting things ready, please wait...” only on the very first message of a session (real model cold-start can take up to ~60-90 seconds), then drop straight to the plain typing dots for every message after. An earlier draft guessed a specific wait time; that was deliberately removed rather than replaced with a shorter, inaccurate estimate.
- **Native `<dialog>` for modals, not a hand-rolled overlay** — Comparator’s review popup uses a real `<dialog>` element (`showModal()`), getting focus-trapping, Esc-to-close, and backdrop dimming for free from the browser instead of reimplementing them in JavaScript.
- **Collapsed-by-default `<details>` for secondary information** — anything useful for double-checking but not needed to act (Comparator’s column-mapping guess) is tucked behind a native `<details><summary>` toggle rather than shown inline, so it doesn’t compete for attention with what actually needs a decision.
- **Plain-language labels over internal/technical ones** — Comparator originally labeled its two sides “SAP/Odoo” and “Second document”; these were replaced with “Our Records” and “Their Statement” everywhere in the review UI, since a small business owner using the tool may not know or care what SAP is.
- **Color as a signal, consistently** — green for confirmed/primary actions, red for errors and genuine mismatches needing attention, amber/orange specifically for weaker-evidence findings that still deserve a second look but shouldn’t be confused with a stronger signal.

11. Known gaps / not yet built

Shown in the UI itself as dimmed, non-clickable “Coming soon” cards, so these are visible product intent, not silent omissions:

- Home page: Multi-Document Synthesis, Voice Assistant
- Translator: translate a file (not just pasted text)
- Converter: PDF-to-DOC, PDF-to-XLSX, scanned-document-to-PDF
- Editor sub-menu: Annotate, Add text boxes, Fill form, Redact

There is also no frontend build/deploy pipeline and no automated frontend test suite yet — every page has been verified by direct manual testing through a real running backend stack, not an automated browser test.